



LAB WORKBOOK

24AD07HF LARGE LANGUAGE MODELS

III B. TECH 2026-27 ODD SEMESTER

K L UNIVERSITY | LARGE LANGUAGE MODELS-24AD07HF



LABORATORY WORKBOOK

STUDENT NAME	
REG. NO	
YEAR	
SEMESTER	
SECTION	
FACULTY	

Table of Contents

Lab Session 01: Statistical Language Modeling using N-Grams	06
Lab Session 02: Learning and Comparing Word Embedding Models.....	11
Lab Session 03: Implementation of Modern Tokenization Techniques.....	16
Lab Session 04: Neural Language Modeling for Next-Word Prediction.....	21
Lab Session 05: Sequence-to-Sequence Modeling and Decoding Methods.....	26
Lab Session 06: Attention Mechanism in Sequence-to-Sequence Models.....	31
Lab Session 07: Implementation of Transformer Encoder Architecture.....	36
Lab Session 08: Exploration of Pre-trained Language Model Architectures.....	41
Lab Session 09: Fine-Tuning Strategies for Language Models.....	46
Lab Session 10: Prompt-Based Learning and Prompt Engineering.....	51
Lab Session 11: Retrieval-Augmented Generation for Question Answering.....	56
Lab Session 12: Knowledge Graph Construction and Representation Learning.....	61
Lab Session 13: Parameter-Efficient Fine-Tuning of Language Models.....	66

2026-27 ODD SEMESTER LAB CONTINUOUS EVALUATION

[illegible]

2026-27 ODD SEMESTER LAB CONTINUOUS EVALUATION

[illegible]

Date of the Session: _____ / _____ / _____

Time of the Session: ____ to _____

LAB 1: Program Title: To implement N-gram Statistical Language Models with Smoothing Techniques and evaluate using Perplexity.

Pre-lab:

1. Read/write about Statistical Language Models, the chain rule of probability, and the Markov assumption underlying N-gram models (unigram, bigram, trigram).
2. Read/write about the zero-probability problem in N-gram models and the need for smoothing. Read about basic smoothing techniques: Laplace (Add-One), Add-k, and Back-off/Interpolation.
3. Read about advanced smoothing algorithms: Good-Turing discounting, Absolute Discounting Interpolation, and Kneser-Ney smoothing. Write down, in your own words, why Kneser-Ney is considered superior to simpler smoothing methods (hint: continuation probability).

In-lab: Using a given text corpus, build N-gram Language Models and apply basic smoothing techniques, then perform the following:

1. Preprocess the corpus (tokenization, sentence boundary markers $\langle s \rangle / \langle /s \rangle$) and build unigram, bigram, and trigram frequency counts.
2. Compute Maximum Likelihood Estimate (MLE) probabilities for bigram/trigram sequences and demonstrate the zero-probability problem on an unseen word sequence.
3. Implement Laplace (Add-One) smoothing and Add-k smoothing; recompute probabilities for the same sequence and compare with raw MLE.
4. Implement Back-off and/or Interpolation and compare smoothed estimates against Add-k.
5. Compute Perplexity of the unigram, bigram, and trigram models (with each basic smoothing method) on a held-out test set (split from the same corpus).
6. Tabulate perplexity values across model order (unigram/bigram/trigram) $\times$ smoothing method (MLE/Laplace/Add-k/Interpolation), and plot perplexity vs. N-gram order.
7. Write a brief report identifying which model+smoothing combination achieves the lowest perplexity among the basic methods tested, and explain why.

Post-lab: Using the same text corpus as the in-lab session, implement advanced smoothing techniques and perform the following:

- (a) Implement Good-Turing discounting and compute revised frequency estimates for low-count N-grams; compare against Add-k smoothing from the in-lab session.
- (b) Implement Kneser-Ney smoothing (or Absolute Discounting Interpolation) using continuation-probability counts, and compute smoothed bigram/trigram probabilities.
- (c) Compute and compare Perplexity of Good-Turing and Kneser-Ney smoothed models against the best basic-smoothing model from the in-lab session, on the same held-out test split.
- (d) Generate sample sentences using the Kneser-Ney smoothed model (via next-word sampling) and qualitatively assess fluency/coherence versus text generated using Add-k smoothing in the in-lab session.
- (e) Identify at least one case where Kneser-Ney handles an unseen/rare N-gram noticeably better than the basic methods, using continuation-count reasoning.
- (f) Save all computed probability tables and perplexity results, and write an observation report comparing basic vs. advanced smoothing performance on the same corpus/test split, and discussing the limitations of statistical N-gram models (data sparsity, lack of long-range context) as motivation for neural language models (RNN/LSTM/GRU) covered next in the course.

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation Marks</u> Secured: _____ out of _____ Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:

LAB 2: Program Title: To construct and compare Word Representations using Word2Vec, fastText, and GloVe.

Pre-lab:

1. Read/write about the distributional hypothesis and why dense word representations are needed over discrete/one-hot encoding.
2. Read/write about how Word2Vec, fastText, and GloVe each learn word embeddings, and note one key difference between them.
3. Read/write about `gensim.downloader` and the pretrained embedding models it provides.

In-lab: Load pretrained Word2Vec, fastText, and GloVe models and compare them on a common set of tasks:

1. Load pretrained Word2Vec (`word2vec-google-news-300`), fastText (`fasttext-wiki-news-subwords-300`), and GloVe (`glove-wiki-gigaword-300`) using `gensim.downloader`.
2. Report vocabulary size and embedding dimension for each model.
3. Run the same word-analogy query (e.g., $\text{king} - \text{man} + \text{woman} \approx \text{queen}$) on all three models and compare results.
4. Retrieve top-5 nearest neighbors for a few common words (e.g., "happy", "computer") from each model and compare.
5. Test a rare/misspelled word on all three models — observe that fastText handles it while Word2Vec and GloVe fail (`KeyError`).
6. Visualize a shared set of ~15–20 words from all three models using t-SNE/PCA.
7. Write a brief report comparing the three models on analogy quality, neighbor quality, and OOV handling.

Post-lab: Using a small set of domain-specific or technical words (e.g., medical or scientific terms):

- (a) Repeat the nearest-neighbor and OOV tests from the In-lab on this new word set.
- (b) Compare how well each of the three pretrained models handles this domain-specific vocabulary.
- (c) Write a short observation report on which model performed best and why, and briefly discuss the limitation of general-purpose pretrained embeddings for specialized domains.

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation Marks</u> Secured: _____ out of _____ Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:

LAB 3: Program Title: To implement and compare Tokenization Strategies: Word-level, BPE, WordPiece, and SentencePiece.

Pre-lab:

1. Read/write about word-level (whitespace/rule-based) tokenization and its limitations (out-of-vocabulary problem, large vocabulary size).
2. Read/write about subword tokenization algorithms — Byte-Pair Encoding (BPE), WordPiece, and SentencePiece (Unigram) — and which models use each (GPT-2 uses BPE, BERT uses WordPiece, T5/XLNet use SentencePiece).
3. Write down why subword tokenization was introduced to solve the OOV problem.

In-lab: Using a given text corpus, perform the following:

1. Tokenize the corpus using simple whitespace/rule-based tokenization; report vocabulary size.
2. Load pretrained tokenizers for BPE (GPT-2), WordPiece (BERT), and SentencePiece (T5 or XLNet) using Hugging Face `transformers/tokenizers`.
3. Tokenize the same corpus using all three subword tokenizers; report resulting vocabulary size and average tokens-per-word for each.
4. Tokenize a few rare/compound/misspelled words with all four methods (whitespace + 3 subword) and compare how each splits them.
5. Tabulate and compare: vocabulary size, tokens-per-word, and OOV handling across all four techniques.
6. Write a brief report on which tokenizer is most efficient/robust for this corpus and why.

Post-lab: Using a domain-specific or morphologically rich piece of text (e.g., medical/technical terms or compound words):

- (a) Tokenize it using all four techniques (whitespace, BPE, WordPiece, SentencePiece).
- (b) Compare how each technique handles the rare/domain-specific words.
- (c) Write a short observation report on which tokenization strategy generalizes best to unseen/rare vocabulary, and why subword methods are preferred in modern LLMs.

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation Marks</u> Secured: _____ out of _____ Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:

LAB 4: Program Title: To implement Neural Language Models using LSTM and GRU for Next-Word Prediction.

Pre-lab:

1. Read/write about the limitations of simple RNNs (vanishing/exploding gradients, inability to capture long-range dependencies) and how LSTM and GRU were designed to address them.
2. Read/write about the LSTM architecture (forget gate, input gate, output gate, cell state) and the GRU architecture (reset gate, update gate) — write down the key structural difference between the two.
3. Read/write about how a neural language model is trained: input embedding layer → recurrent layer → output softmax over vocabulary, predicting the next token given previous tokens.

In-lab: Using a given text corpus, build and train LSTM and GRU based language models:

1. Preprocess the corpus (tokenize, build vocabulary, convert text to sequences of token IDs) and prepare fixed-length input-target sequences for next-word prediction (e.g., using a sliding window).
2. Build an embedding layer followed by an LSTM layer and a final linear/softmax layer for next-word prediction (PyTorch `nn.LSTM`).
3. Train the LSTM-based language model on the corpus; plot training loss over epochs.
4. Build the same architecture but replace the LSTM layer with a GRU layer (PyTorch `nn.GRU`); train it on the same data under identical settings (same epochs, learning rate, hidden size).
5. Compare LSTM vs GRU on: training loss curves, training time per epoch, and number of trainable parameters.
6. Generate text/next-word predictions from both trained models given a seed sequence, and qualitatively compare fluency/coherence.
7. Write a brief report comparing LSTM and GRU on convergence speed, computational cost, and output quality.

Post-lab: Using a different (larger or domain-specific) text corpus:

- (a) Train both LSTM and GRU language models under the same hyperparameter settings.
- (b) Compute and compare perplexity of both models on a held-out test set.
- (c) Generate sample text from both models using the same seed sequence and compare output fluency.
- (d) Write an observation report on which model (LSTM or GRU) performed better on this corpus, discussing the trade-off between model complexity and performance, and connecting this to why Transformer-based architectures (covered next) eventually replaced RNN-based LMs for most tasks.

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation Marks</u> Secured: _____ out of _____ Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:

LAB 5: Program Title: To implement Sequence-to-Sequence Models and compare Greedy Decoding with Beam Search.

Pre-lab:

1. Read/write about the Sequence-to-Sequence (Seq2Seq) architecture — encoder-decoder framework — and its typical applications (machine translation, summarization, text generation).
2. Read/write about Greedy Decoding (always selecting the highest-probability next token) and its limitations (short-sighted, can miss globally better sequences).
3. Read/write about Beam Search decoding — maintaining multiple candidate sequences (beams) at each step — and how beam width affects the search.

In-lab: Using a given Seq2Seq model (either trained from scratch on a small dataset or a pretrained model such as GPT-2/T5 via Hugging Face), perform the following:

1. Build/load a Seq2Seq model (encoder-decoder architecture using LSTM/GRU, or a pretrained model) for a text generation or translation-style task.
2. Implement Greedy Decoding: at each decoding step, select the token with the highest probability, and generate a complete output sequence for a given input.
3. Implement Beam Search decoding with a chosen beam width (e.g., beam width = 3); generate output sequences and compare against Greedy Decoding.
4. Vary the beam width (e.g., 2, 5, 10) and observe changes in output quality, diversity, and computation time.
5. Compare Greedy Decoding vs Beam Search outputs on: fluency, repetition, and relevance to the input.
6. Tabulate output sequences and their log-probability scores across Greedy Decoding and different beam widths.
7. Write a brief report on the trade-offs between Greedy Decoding and Beam Search (quality vs computational cost).

Post-lab: Using the same model, implement and compare Advanced Decoding Techniques — Temperature Sampling, Top-k Sampling, and Nucleus (Top-p) Sampling:

(a) Implement Temperature Sampling with varying temperature values (e.g., 0.5, 1.0, 1.5) and observe changes in output randomness/diversity.

(b) Implement Top-k Sampling with varying k values (e.g., k=5, 20, 50) and compare output diversity and coherence.

(c) Implement Nucleus (Top-p) Sampling with varying p values (e.g., p=0.8, 0.9, 0.95) and compare against Top-k Sampling.

(d) Generate multiple outputs for the same input prompt using each advanced decoding technique and qualitatively assess diversity, repetition, and fluency.

(e) Write an observation report comparing Greedy Decoding, Beam Search, Temperature Sampling, Top-k Sampling, and Nucleus Sampling overall — discussing which decoding strategy is best suited for which downstream application (e.g., factual QA vs. creative text generation).

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation Marks</u> Secured: _____ out of _____ Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:

LAB 6: Program Title: To implement Attention Mechanism in Sequence-to-Sequence Models and visualize Attention Weights.

Pre-lab:

1. Read/write about the limitation of a basic Seq2Seq model — the encoder must compress the entire input sequence into a single fixed-length context vector, which becomes a bottleneck for long sequences.
2. Read/write about the Attention mechanism (Bahdanau/Additive Attention and Luong/Multiplicative Attention) and how it allows the decoder to "look back" at all encoder hidden states at each decoding step, rather than relying on a single fixed vector.
3. Write down, in your own words, how attention weights are computed (alignment scores $\rightarrow$ softmax $\rightarrow$ weighted sum of encoder states) and what they represent.

In-lab: Using a given Seq2Seq task (e.g., sequence reversal, sorting, or a small translation-style task), build and train an attention-augmented Seq2Seq model:

1. Build a baseline Seq2Seq (Encoder-Decoder, no attention) model and note its performance/limitation on longer input sequences.
2. Add an Attention layer (Bahdanau or Luong style) between the encoder and decoder; modify the decoder to compute alignment scores, attention weights, and a context vector at each decoding step.
3. Train the attention-augmented Seq2Seq model on the same task/dataset used for the baseline.
4. Compare training loss and output accuracy of the attention-based model against the no-attention baseline, particularly on longer input sequences.
5. Extract and visualize the attention weight matrix for a few sample input-output pairs as a heatmap (input tokens vs. output tokens).
6. Analyze the attention heatmap: verify whether the model attends to the correct/expected input position(s) when generating each output token.
7. Write a brief report comparing baseline Seq2Seq vs. attention-augmented Seq2Seq, and describing what the attention heatmaps reveal about the model's alignment behavior.

Post-lab: Using a longer/more complex sequence task or dataset (e.g., longer sequences, or a small English-to-Pig-Latin or word-reversal-with-noise task):

- (a) Train both baseline and attention-augmented Seq2Seq models on this new, longer-sequence task.
- (b) Compare performance degradation of the no-attention baseline vs. the attention model as sequence length increases.
- (c) Visualize attention heatmaps for a few longer examples and identify any misalignment patterns.
- (d) Write an observation report on how attention mitigates the fixed-context-vector bottleneck, and how this mechanism directly motivates the fully attention-based Transformer architecture (Self-Attention and Multi-Head Attention) covered next in the course.

Comment of the Evaluator (if Any)

Evaluator's Observation Marks

Secured: _____ out of _____

Full Name of the Evaluator:

Signature of the Evaluator Date of Evaluation:

LAB 7: Program Title: To implement a Transformer Encoder Block from Scratch — Self-Attention, Multi-Head Attention, Positional Encoding, and Layer Normalization.

Pre-lab:

1. Read/write about the limitation of RNN/LSTM-based sequence models (sequential computation, difficulty capturing long-range dependencies) and how Transformers process sequences in parallel using self-attention instead of recurrence.
2. Read/write about Scaled Dot-Product Self-Attention — Query, Key, Value matrices, the attention score formula and write down what each component represents.
3. Read/write about Multi-Head Attention — why splitting attention into multiple heads allows the model to attend to information from different representation subspaces simultaneously.
4. Read/write about Positional Encoding (sinusoidal) and why it's needed — Transformers have no inherent notion of token order since self-attention is permutation-invariant.
5. Read/write about Layer Normalization and residual connections, and their role in stabilizing training in deep Transformer stacks.

In-lab: Implement each Transformer component from scratch in PyTorch (without using `nn.MultiheadAttention` or `nn.TransformerEncoderLayer`), then assemble a complete Transformer Encoder block:

1. Implement Scaled Dot-Product Self-Attention from scratch on a toy input sequence; verify output shape and inspect the resulting attention weight matrix.
2. Implement Multi-Head Attention by splitting Q/K/V into multiple heads, applying self-attention independently per head, and concatenating the results back together.
3. Implement sinusoidal Positional Encoding and add it to token embeddings; visualize the positional encoding matrix as a heatmap to observe its periodic structure.
4. Implement Layer Normalization and a residual (skip) connection around the Multi-Head Attention sub-layer.
5. Assemble a complete Transformer Encoder block: Multi-Head Attention → Add & Norm → Feed-Forward Network → Add & Norm.
6. Validate your from-scratch Multi-Head Attention output against PyTorch's built-in `nn.MultiheadAttention` (same input, same weights) to confirm numerical correctness.
7. Train your assembled Encoder block on a toy sequence task (e.g., copy task or sequence classification) to confirm it can learn.
8. Write a brief report explaining each component's role and your validation results.

Post-lab: Using a different toy task (e.g., sequence reversal or a simple parity/counting task):

- (a) Stack two Transformer Encoder blocks (from your from-scratch implementation) and train on this new task.
- (b) Visualize the self-attention weight matrix from each layer for a sample input, and compare attention patterns between layer 1 and layer 2.
- (c) Compare training performance (loss, accuracy) of your 2-layer Transformer Encoder against the single-layer version from the In-lab.
- (d) Write an observation report on how attention patterns differ across layers, and how this from-scratch understanding connects to pretrained Transformer-based models (BERT/GPT) covered next in the course.

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation Marks</u> Secured: _____ out of _____ Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:

LAB 8: Program Title: To examine Pre-Training Strategies and distinguish Encoder-only, Encoder-Decoder, and Decoder-only Language Models.

Pre-lab:

1. Read/write about pretraining objectives: ELMo (bidirectional LSTM-based contextual embeddings via language modeling), BERT (Masked Language Modeling + Next Sentence Prediction), and autoregressive pretraining (GPT-style).
2. Read/write about the three Transformer architecture families: encoder-only (BERT), encoder-decoder (T5/BART), and decoder-only (GPT) — and which pretraining objective/task each is best suited for.
3. Write down the key difference between ELMo's contextual embeddings and BERT's contextual embeddings (hint: LSTM vs Transformer, shallow concatenation vs deep bidirectional attention).

In-lab:

1. Load a pretrained ELMo model (via AllenNLP/TensorFlow Hub or a similar library) and extract contextual embeddings for a polysemous word (e.g., "bank") in two different sentences; compute cosine similarity between the two occurrences.
2. Load a pretrained BERT model and repeat the same polysemous-word experiment; compare BERT's contextual sensitivity against ELMo's.
3. Load BERT (encoder-only), T5 (encoder-decoder), and GPT-2 (decoder-only) pretrained models; pass the same input through each and inspect output type/shape (embeddings vs. generated text) and attention masking pattern (bidirectional vs. causal).
4. Run a task suited to each architecture: BERT for masked-token prediction (fill-in-the-blank), T5 for a text-to-text reformulation task, GPT-2 for free-form continuation.
5. Tabulate architectural differences: attention masking, pretraining objective, typical downstream use case, for all four models (ELMo, BERT, T5, GPT-2).
6. Write a brief report comparing the four models' contextual behavior and architecture suitability.

Post-lab: Using a different polysemous word and a different downstream task per architecture:

- (a) Repeat the contextual embedding comparison (ELMo vs. BERT) on the new word.
- (b) Run the new task through BERT, T5, and GPT-2 respectively.
- (c) Write an observation report on which architecture would be most suitable for a given real-world application (e.g., search/classification → encoder-only; translation/summarization → encoder-decoder; open-ended generation → decoder-only).

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation Marks</u> Secured: _____ out of _____ Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:

LAB 9: Program Title: To compare Full Fine-Tuning and Instruction Tuning for downstream NLP task adaptation.

Pre-lab:

1. Read/write about the standard fine-tuning paradigm (task-specific fine-tuning of a pretrained model on labeled data) versus instruction tuning (training a model on a diverse mixture of tasks phrased as natural-language instructions, e.g., FLAN, T0, InstructGPT).
2. Read/write about why instruction-tuned models can perform new tasks in a zero-shot manner without task-specific fine-tuning.
3. Write down the key trade-off between full fine-tuning (task-specific, requires labeled data per task) and instruction tuning (general-purpose, zero-shot capable, but may underperform a fully fine-tuned specialist model).

In-lab:

1. Fine-tune a pretrained BERT/DistilBERT model on a specific downstream task (e.g., sentiment classification) using the Hugging Face Trainer API; record accuracy/F1.
2. Load an instruction-tuned model (e.g., Flan-T5) and evaluate it zero-shot on the same task by phrasing the input as a natural-language instruction (e.g., "Classify the sentiment of this review: ...").
3. Compare accuracy of the fully fine-tuned BERT model vs. the zero-shot instruction-tuned model on the same test set.
4. Evaluate the instruction-tuned model on a second, different task (e.g., summarization or question answering) without any additional fine-tuning, to demonstrate its multi-task generalization.
5. Tabulate accuracy/quality of fine-tuned vs. instruction-tuned model across both tasks.
6. Write a brief report on when full fine-tuning is worth the additional data/compute cost versus when a zero-shot instruction-tuned model is sufficient.

Post-lab: Using a different downstream task (e.g., topic classification or NLI):

(a) Fine-tune BERT/DistilBERT on this new task.

(b) Evaluate the same instruction-tuned model zero-shot on this task.

(c) Compare performance and discuss whether the gap between fine-tuned and zero-shot performance widens or narrows compared to the In-lab task, and why.

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation Marks</u> Secured: _____ out of _____ Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:

LAB 10: Program Title: To implement Prompt-based Learning and analyze Advanced Prompting Techniques and Prompt Sensitivity.

Pre-lab:

1. Read/write about prompt-based learning and in-context learning — Zero-shot, One-shot, and Few-shot prompting formats.
2. Read/write about advanced prompting techniques: Chain-of-Thought (CoT) prompting and Self-Consistency, and how they improve performance on reasoning tasks.
3. Read/write about prompt sensitivity — how paraphrasing or reordering a prompt can change model output — and why this is considered a limitation of current LLMs.

In-lab:

1. Using an LLM (local small model or API-based), design Zero-shot, One-shot, and Few-shot prompts for a classification or QA task; record and compare accuracy across the three formats.
2. Design a Chain-of-Thought prompt (e.g., "Let's think step by step") for a multi-step reasoning task (arithmetic word problem or logical puzzle) and compare output correctness against a direct (non-CoT) prompt.
3. Implement Self-Consistency: sample multiple CoT outputs for the same question (using temperature > 0) and select the majority-vote final answer; compare accuracy against a single CoT sample.
4. Test prompt sensitivity: create 3–4 paraphrased versions of the same instruction/prompt and record output variation across them.
5. Test few-shot example ordering sensitivity: reorder the same few-shot examples within a prompt and observe if output changes.
6. Tabulate accuracy/consistency across all prompting techniques and paraphrase variations.
7. Write a brief report on which prompting technique performed best for which task type, and how sensitive the model was to prompt phrasing/ordering.

Post-lab: Using a different reasoning/QA task:

- (a) Repeat the Zero-shot/One-shot/Few-shot comparison.
- (b) Repeat the CoT vs. Self-Consistency comparison.
- (c) Test prompt sensitivity with a new set of paraphrased instructions.
- (d) Write an observation report summarizing which prompting strategy is most robust across tasks, and discuss the implications of prompt sensitivity for reliable real-world LLM deployment.

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation</u> Marks Secured: _____ out of _____ Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:

LAB-11: Program Title: To implement Retrieval-Augmented Generation (RAG) for Open-book Question Answering from Structured and Unstructured Sources.

Pre-lab:

1. Read/write about closed-book vs. open-book QA, and how RAG combines a retriever with a generator to ground LLM answers in external knowledge.
2. Read/write about retrieval over unstructured sources (documents, embedded via sentence-transformers, indexed in a vector database like FAISS/Chroma) versus retrieval over structured sources (tables, knowledge graphs, SQL databases queried directly or via text-to-SQL).
3. Read/write about retrieval augmentation techniques: dense retrieval (embedding similarity), sparse retrieval (BM25/keyword-based), and hybrid retrieval (combining both); write down the trade-offs of each.

In-lab: Build two RAG pipelines — one over unstructured text, one over structured data — and compare retrieval techniques:

1. Unstructured RAG: Chunk a small document corpus, embed chunks using sentence-transformers, index in FAISS, and build a retrieval + generation pipeline (retrieve top-k chunks → generate answer using an LLM with retrieved context).
2. Structured RAG: Given a small structured dataset (a table/CSV or simple knowledge base of triples), implement retrieval via direct querying (e.g., pandas filtering, SQL query, or triple lookup) and feed the retrieved structured result into the LLM as context to generate a natural-language answer.
3. Evaluate both pipelines on a set of open-book questions matched to their respective source (document-based questions for unstructured RAG, tabular/fact-based questions for structured RAG).
4. Compare dense retrieval (embedding similarity) vs. sparse retrieval (BM25) on the unstructured RAG pipeline; measure retrieval quality (e.g., whether the correct chunk was retrieved in top-k) and downstream answer correctness.
5. Compare answer quality/groundedness of RAG-generated answers against the same LLM answering without retrieval (closed-book), for both structured and unstructured sources.
6. Tabulate retrieval technique (dense/sparse/hybrid) × answer accuracy, and structured vs. unstructured × answer accuracy.
7. Write a brief report on which retrieval technique performed best for which source type, and how retrieval augmentation reduced hallucination compared to closed-book generation.

Post-lab: Using a different document corpus and structured dataset (e.g., a different domain):

- (a) Rebuild both RAG pipelines (unstructured + structured) on the new data.
- (b) Implement a simple hybrid retrieval (combine dense + sparse scores) and compare against pure dense/sparse retrieval from the In-lab.
- (c) Vary the top-k retrieval count and observe its effect on answer quality for both source types.
- (d) Write an observation report on which retrieval augmentation technique generalizes best across domains, and discuss practical considerations for choosing structured vs. unstructured retrieval strategies in a real-world RAG system.

<u>Comment of the Evaluator (if Any)</u>	<u>Evaluator's Observation Marks</u> Secured: _____ out of _____ Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:

LAB-12: Program Title: To construct Knowledge Graphs and implement Representation, Completion, Alignment, and Isomorphism Techniques.

Pre-lab:

1. Read/write about Knowledge Graph representation (entities, relations, triples: subject-predicate-object) and common KG embedding models (TransE, DistMult).
2. Read/write about Knowledge Graph Completion (link prediction) — predicting missing triples/relations in an incomplete KG.
3. Read/write about Knowledge Graph Alignment (entity alignment across two different KGs referring to overlapping real-world entities) and Graph Isomorphism (structural equivalence between two graphs).

In-lab:

1. Construct a small Knowledge Graph from a text corpus (entity/relation extraction using spaCy or an LLM-based extractor) and represent it as a set of triples; visualize the graph using `networkx`.
2. Train a KG embedding model (TransE or DistMult) on the constructed triples using a library such as PyKEEN.
3. Perform Knowledge Graph Completion: given a partial triple (e.g., subject + relation, missing object), use the trained embedding model to predict the most likely missing entity; evaluate using Mean Reciprocal Rank (MRR) or Hits@k.
4. Construct a second, related Knowledge Graph (e.g., from a different but overlapping text source) and identify a few candidate entity pairs referring to the same real-world entity across both graphs.
5. Perform simple Knowledge Graph Alignment: compute embedding similarity between candidate entity pairs across the two KGs to identify likely alignments.
6. Check Graph Isomorphism between two small subgraphs (e.g., using `networkx.is_isomorphic`) and discuss what structural equivalence means in the KG context.
7. Write a brief report on completion accuracy, alignment quality, and isomorphism findings.

Post-lab: Using a different text corpus (e.g., a different domain, such as movies or sports):

- (a) Construct a new Knowledge Graph and train a KG embedding model on it.
- (b) Perform link prediction/KG completion and evaluate using MRR/Hits@k.
- (c) Compare completion performance against the In-lab KG — discuss whether KG size/density affected performance.
- (d) Write an observation report on the challenges of KG construction from unstructured text (extraction errors, relation ambiguity) and their downstream effect on completion/alignment quality.

Comment of the Evaluator (if Any)

Evaluator's Observation Marks

Secured: _____ out of _____

Full Name of the Evaluator:

Signature of the Evaluator Date of Evaluation:

LAB-13: Program Title: To implement Parameter-Efficient Fine-Tuning (Prompt Tuning, Prefix Tuning, LoRA) and analyze Language Models via the Residual Stream Perspective.

Pre-lab:

1. Read/write about why full fine-tuning of large LMs is computationally expensive, and how Parameter-Efficient Fine-Tuning (PEFT) methods address this by training only a small number of additional parameters.
2. Read/write about Prompt Tuning (learnable soft prompt embeddings prepended to input), Prefix Tuning (learnable prefix vectors injected into each Transformer layer's keys/values), and LoRA (low-rank decomposition of weight updates).
3. Read/write about the residual stream perspective of Transformers — viewing each layer as reading from and writing to a shared residual stream, which provides a unifying lens for interpretability (used in mechanistic interpretability research).

In-lab:

1. Load a small pretrained LM and a downstream task dataset (e.g., sentiment classification).
2. Fine-tune using LoRA (via Hugging Face `peft` library); record trainable parameter count, training time, and task accuracy.
3. Fine-tune using Prompt Tuning; record the same metrics.
4. Fine-tune using Prefix Tuning; record the same metrics.
5. Tabulate and compare all three PEFT methods against each other on: trainable parameter count (as % of full model), training time, and accuracy.
6. Residual stream analysis: Using hooks, extract the residual stream's hidden state after each Transformer layer for a sample input; compute the norm of the residual stream at each layer and plot how it grows across depth.
7. Use a simple logit lens technique: project the residual stream at each intermediate layer through the model's final unembedding matrix to see what token the model would predict "if it stopped here" — observe how the predicted token evolves from early to late layers.
8. Write a brief report comparing the three PEFT methods, and describing what the residual stream/logit lens analysis reveals about how the model builds its prediction across layers.

Post-lab: Using a different downstream task (e.g., a different classification dataset or NER):

- (a) Apply LoRA, Prompt Tuning, and Prefix Tuning adaptation to this new task.
- (b) Compare performance-to-trainable-parameter trade-off across all three methods.
- (c) Repeat the residual stream norm growth and logit lens analysis for a model fine-tuned on this task; compare with the In-lab findings.
- (d) Write an observation report on which PEFT method is most efficient for this task, and what the residual stream perspective reveals about how fine-tuning changes (or doesn't change) internal layer-wise predictions compared to the base pretrained model.

Comment of the Evaluator (if Any)

Evaluator's Observation Marks

Secured: _____ out of _____

Full Name of the Evaluator:

Signature of the Evaluator Date of Evaluation: